

Baligh

Arabic Writing Assistant

A Graduation Project Report Submitted

to

Faculty of Engineering, Cairo University

in Partial Fulfillment of the requirements of the degree

of

Bachelor of Science in Computer Engineering.

Presented by

Ahmed Hamed Gaber Hamed

Akram Hany Karam Salam

Amir Anwar Bekhit Awd Kedis

Somia Saad Ismail El-Shemy

Supervised by

Dr. Ayman AboElhassan

July, 2025

All rights reserved. This report may not be reproduced in whole or in part, by photocopying or other means, without the permission of the authors/department.

Abstract

Arabic digital writing still suffers from a shortage of integrated tools that can correct Modern Standard Arabic text accurately, respond in real time, and explain their feedback in a way that helps users learn. Existing tools often focus on surface spelling fixes or black-box suggestions without exposing the grammatical reasoning behind the correction. Baligh addresses this problem by providing an Arabic writing assistant that combines grammatical error detection, grammatical error correction, spelling-aware editing, next-word suggestion, and explanatory feedback in a single user-facing system. The project aims to support students, educators, and Arabic content writers by improving writing quality while also making the correction process more understandable and educational.

The implemented system follows a modular pipeline. A preprocessing layer performs normalization, word-boundary handling, segmentation, and morphological analysis and disambiguation. On top of that, Baligh integrates a grammatical error detection engine that fuses rule-based, lexicon-based, and machine-learning detectors, then passes its findings to correction components built around ontology-guided reasoning, dictionary lookup, and text-editing models. In parallel, a next-word suggestion module supports both next-word prediction and word auto-completion. The final system highlights issues, proposes corrections, returns ranked suggestions, and presents grammar-oriented explanations to the user in an interactive workflow.

The project outputs include the preprocessing pipeline, grammatical error detection and correction services, the next-word suggestion subsystem, and a working user application. Testing and validation relied on automated unit and integration tests together with evaluation on established Arabic linguistic resources, including QALB14, QALB15, and ZAEBUC. Overall, Baligh demonstrates that a modular and explainable Arabic writing assistant can combine linguistic processing and machine learning into a practical platform for improving Arabic writing quality.

الملخص

لا تزال الكتابة الرقمية باللغة العربية تعاني من نقص الأدوات المتكاملة التي تستطيع تصحيح نصوص العربية الفصحى بدقة، والعمل بزمان استجابة مناسب، وتقديم تفسير واضح يساعد المستخدم على التعلم بدل الاكتفاء بإظهار التصحيح فقط. فكثير من الأدوات المتاحة تركز على الأخطاء الإملائية السطحية أو تقدم اقتراحات غير مفسرة. يعالج مشروع بليغ هذه المشكلة من خلال بناء مساعد كتابة عربي يجمع بين اكتشاف الأخطاء النحوية، وتصحيحها، ومعالجة الأخطاء الإملائية، واقتراح الكلمة التالية، وتقديم تفسير مبسط للقاعدة أو سبب الخطأ داخل نظام واحد موجه للمستخدم. ويهدف المشروع إلى خدمة الطلاب والمعلمين وصناع المحتوى العربي عبر تحسين جودة الكتابة وجعل عملية التصحيح أكثر فائدة من الناحية التعليمية.

يعتمد النظام المنفذ على بنية معيارية تبدأ بطبقة للمعالجة المسبقة تشمل التطبيع، وتحديد حدود الكلمات، والتجزئة، ثم التحليل الصرفي وفك الالتباس. بعد ذلك يدمج بليغ محركاً لاكتشاف الأخطاء النحوية يعتمد على المزج بين قواعد لغوية، وكشافات معجمية، ونماذج تعلم آلي، ثم يمرر النتائج إلى وحدات التصحيح المبنية على الاستدلال المعتمد على الأنطولوجيا، والبحث المعجمي، ونماذج التحرير النصي. وبالتوازي مع ذلك، يوفر النظام وحدة لاقتراح الكلمة التالية تشمل التنبؤ بالكلمة التالية والإكمال التلقائي للكلمة الجارية كتابتها. وتعرض المنصة النهائية الأخطاء والتصحيحات والاقتراحات والتفسيرات اللغوية بصورة تفاعلية.

تشمل مخرجات المشروع خط معالجة مسبقة متكامل، وخدمات لاكتشاف الأخطاء وتصحيحها، ووحدة اقتراح الكلمات، وتطبيقاً عملياً للمستخدم. واعتمد التحقق من النظام على اختبارات آلية للوحدات والتكامل، إضافة إلى تقييم مبني على موارد عربية معروفة مثل QALB14 و QALB15 و ZAEUC. وتظهر المحصلة أن بليغ يقدم نموذجاً عملياً لمساعد كتابة عربي قابل للتفسير يجمع بين المعالجة اللغوية والتعلم الآلي لخدمة الكتابة العربية بصورة أكثر جودة وفائدة.

Acknowledgment

Baligh was not built through technical effort alone, but through the support, patience, and trust of many people around us. We would first like to thank our supervisor, Dr. Ayman AboElhassan, for his steady guidance throughout this project. His questions, feedback, and encouragement pushed us to think more carefully, work more seriously, and keep improving the project whenever we felt uncertain or stuck.

We are also deeply grateful to the Faculty of Engineering at Cairo University and the Department of Computer Engineering. The years we spent there shaped the way we think, learn, and solve problems, and this project reflects much of what we gained from that journey. Baligh became possible because of the academic foundation and sense of responsibility that this environment gave us.

We would also like to thank one another as a team. This project demanded time, patience, repeated trial and error, and many long discussions before ideas became clear and features became real. What made that process meaningful was not only the final result, but the way we supported each other through the difficult parts, shared the pressure, and kept moving forward together.

Finally, we thank our families and loved ones with all sincerity. Their patience during busy and stressful periods, their constant encouragement, and their belief in us gave us the strength to continue. Any achievement in this work carries a part of their support within it, and for that we are truly grateful.

Contents

Abstract	i
المُلخَص	ii
Acknowledgment	iii
List of Figures	v
List of Tables	vi
List of Abbreviations	vii
List of Symbols	viii
Contacts	ix
1 Introduction	1
1.1 Motivation and Justification	1
1.2 Project Objectives and Problem Definition	1
1.3 Project Outcomes	2
1.4 Document Organization	3
2 Market Visibility Study	4
2.1 Targeted Customers	4
2.2 Market Survey	4
2.2.1 Competitive Product 1	4
2.2.2 Competitive Product 2	4
2.3 Business Case and Financial Analysis	4
3 Literature Survey	6
3.1 Background on Arabic NLP	6
3.2 Background on Writing Assistance Systems	6
3.3 Comparative Study of Previous Work	6
3.4 Implemented Approach	6

4	System Design and Architecture	7
4.1	Overview and Assumptions	7
4.2	System Architecture	7
4.2.1	Block Diagram	7
4.3	Module 1: Grammatical Error Detection	7
4.3.1	Functional Description	7
4.3.2	Modular Decomposition	8
4.3.3	Design Constraints	8
4.3.4	Other Description of Module 1	8
4.4	Module 2: Correction and Explanation Generation	8
4.4.1	Functional Description	8
4.4.2	Modular Decomposition	8
4.4.3	Design Constraints	8
4.4.4	Other Description of Module 2	8
5	System Testing and Verification	9
5.1	Testing Setup	9
5.2	Testing Plan and Strategy	10
5.2.1	Module Testing	10
5.2.2	Integration Testing	11
5.3	Test Schedule	11
5.4	Comparative Results to Previous Work	12
5.4.1	Grammatical Error Detection (GED) Performance	12
5.4.2	Next-Word Prediction (NWP) Performance	14
5.4.3	Word Auto-Completion (WAC) Performance	14
6	Conclusions and Future Work	15
6.1	Faced Challenges	15
6.2	Gained Experience	15
6.3	Conclusions	15
6.4	Future Work	15
A	Development Platforms and Tools	17
B	Use Cases	18
C	User Guide	19
D	Code Documentation	20
E	Feasibility Study	21

List of Figures

4.1 High-level architecture of the Baligh system. 7

List of Tables

- 2.1 Sample Five-Year Financial Projection 5
- 3.1 Comparison of Related Arabic Writing Assistance Approaches 6
- 5.1 Grammatical Error Detection Subsystem Performance 13
- 5.2 Next-Word Prediction (NWP) Accuracy Comparison 14
- 5.3 WAC Top-5 Accuracy by Typed Prefix Length 14

List of Abbreviations

Put abbreviations in alphabetical order.

Abbreviation	Meaning
AI	Artificial Intelligence
GEC	Grammatical Error Correction
GED	Grammatical Error Detection
MSA	Modern Standard Arabic
NLP	Natural Language Processing
NWP	Next-Word Prediction

List of Symbols

Put symbols in alphabetical order. Greek symbols should come first, followed by English symbols.

Symbol	Meaning
α	Learning rate or weighting coefficient
P	Probability
T	Time or sequence length

Contacts

Team Members

Name	Email	Phone Number
Ahmed Hamed Gaber Hamed	ahmed.hamed03@eng-st.cu.edu.eg	01226968657
Akram Hany Karam Salam	akram.sallam03@eng-st.cu.edu.eg	01148746563
Amir Anwar Bekhit Awd Kedis	amir.awd03@eng-st.cu.edu.eg	01221461992
Somia Saad Ismail El-Shemy	somia.elshemy02@eng-st.cu.edu.eg	01080732462

Supervisor

Name	Email	Phone Number
Dr. Ayman AboElhassan	ayman.abo.elmaaty@eng.cu.edu.eg	01060836189

Chapter 1: Introduction

Baligh is an Arabic writing assistant designed to support Modern Standard Arabic writing through explainable correction and real-time assistance. This chapter introduces the motivation behind the project, defines the problem it addresses, summarizes the main outcomes delivered by the project, and outlines the organization of the remainder of the report.

1.1. Motivation and Justification

Arabic digital writing still lacks broadly available tools that combine grammatical support, writing speed, and educational value in a single user experience. While many writing assistants for other languages provide immediate correction, prediction, and revision support, Arabic users often rely on tools that address only isolated spelling mistakes, provide limited grammatical help, or return suggestions without clarifying the linguistic reason behind them. This gap is especially important in Modern Standard Arabic, where accurate writing is influenced by rich morphology, orthographic ambiguity, and context-sensitive grammatical structure.

These characteristics make Arabic NLP more demanding to model and more difficult to serve well in real time. A useful writing assistant must do more than detect surface-level errors; it must process text carefully, understand partially typed input, and return feedback quickly enough to support natural writing. It must also present corrections in a way that is understandable to users rather than functioning as a black box.

Baligh is motivated by this need for an integrated Arabic writing assistant that combines grammatical error detection, grammatical error correction, next-word assistance, and explanatory feedback in one workflow. The project is intended to benefit students, educators, and Arabic content writers who need writing support that improves both text quality and user understanding at the same time.

1.2. Project Objectives and Problem Definition

The primary objective of Baligh is to build an explainable writing assistant for Modern Standard Arabic that can help users write more accurately and efficiently. To achieve this objective, the project integrates a preprocessing pipeline for normalization,

segmentation, and morphological analysis with a grammatical error detection subsystem, grammatical error correction components, a next-word suggestion module, and a user-facing application interface. The system is designed to highlight issues, generate ranked suggestions, provide next-word and auto-completion support, and present explanations that make the feedback more useful for learning.

In addition to functional assistance, the project aims to demonstrate that Arabic writing support can be delivered through a modular architecture that balances linguistic processing, machine learning, usability, and responsiveness. This makes Baligh not only a correction tool, but also a platform for combining multiple Arabic NLP capabilities into a practical workflow.

Formally, the problem addressed by this project is the lack of an integrated and explainable system for Modern Standard Arabic writing that can detect and correct grammatical issues, assist users during text entry, and deliver reliable feedback in a form suitable for real-time interactive use.

1.3. Project Outcomes

The project produced a working set of interconnected software components that together form the Baligh writing assistant. The implemented outcomes include a preprocessing pipeline responsible for text normalization, word-boundary handling, segmentation, and morphological analysis, as well as a grammatical error detection subsystem that combines rule-based, lexicon-based, and machine-learning detectors through a fusion layer.

The project also delivered grammatical error correction components that use ontology-based reasoning, dictionary-based lookup, and text-editing approaches to generate and rank correction candidates. In parallel, Baligh includes a next-word suggestion subsystem that supports both next-word prediction and word auto-completion for interactive writing scenarios.

At the application level, the project provides a backend API and a web-based client that expose these capabilities to the user through an integrated writing workflow. Testing and evaluation were supported through automated checks and experiments on established Arabic resources, including QALB14, QALB15, and ZAEUC, in addition to the accompanying project documents and technical materials.

1.4. Document Organization

The remainder of this report is organized as follows. Chapter 2 presents the Market Visibility Study, which examines the intended users of the system, reviews related products, and discusses the practical need for Baligh.

Chapter 3 provides the Literature Survey, covering the Arabic NLP background and the research context most relevant to the project. Chapter 4 describes the System Design and Architecture, including the main modules, their responsibilities, and the overall data flow.

Chapter 5 discusses System Testing and Verification, explaining how the implemented components were checked and evaluated. Finally, Chapter 6 presents the Conclusions and Future Work, summarizing the project contributions and outlining possible directions for extending Baligh in later stages.

Chapter 2: Market Visibility Study

This chapter surveys the market related to the project and discusses similar tools or platforms. It should explain their limitations and justify the need for Baligh.

2.1. Targeted Customers

Mention the intended customers/users and how they benefit from the project.

2.2. Market Survey

List competing or related products. Discuss each one in a subsection and explain its pros and cons.

2.2.1. Competitive Product 1

Describe the product, strengths, weaknesses, and relevance to Baligh.

2.2.2. Competitive Product 2

Describe the product, strengths, weaknesses, and relevance to Baligh.

2.3. Business Case and Financial Analysis

Describe the possible business case, expected users, pricing assumptions, CapEx, OpEx, revenue, cash flow, and break-even point.

Table 2.1: Sample Five-Year Financial Projection

Year	Expected Users	Revenue	Expenses	Profit Before Tax
1	—	—	—	—
2	—	—	—	—
3	—	—	—	—
4	—	—	—	—
5	—	—	—	—

Chapter 3: Literature Survey

This chapter should include necessary engineering and non-engineering background, followed by a concise literature review of recent work related to the project.

3.1. Background on Arabic NLP

Explain the most important concepts needed to understand the project, such as morphology, spelling correction, grammatical error correction, and language modeling.

3.2. Background on Writing Assistance Systems

Explain writing assistants, correction pipelines, feedback generation, and user-facing explanations.

3.3. Comparative Study of Previous Work

Classify and compare previous work. Focus on relevance, strengths, limitations, datasets, methods, and results.

Table 3.1: Comparison of Related Arabic Writing Assistance Approaches

Work/System	Main Approach	Strengths	Limitations
System 1	Rule-based	Explainable	Limited coverage
System 2	Neural	Strong accuracy	Less interpretable
System 3	Hybrid	Balanced	Requires integration effort

3.4. Implemented Approach

State the approach chosen from the reviewed approaches and justify why it was selected. Mention any modifications added by the team.

Chapter 4: System Design and Architecture

This chapter is the main body of the report. It should answer: what has been done, and how has it been done? The discussion should flow from the overall architecture to modules and submodules.

4.1. Overview and Assumptions

Introduce the system design and list the assumptions used during design and implementation.

4.2. System Architecture

Describe the overall architecture and how modules communicate.

4.2.1. Block Diagram

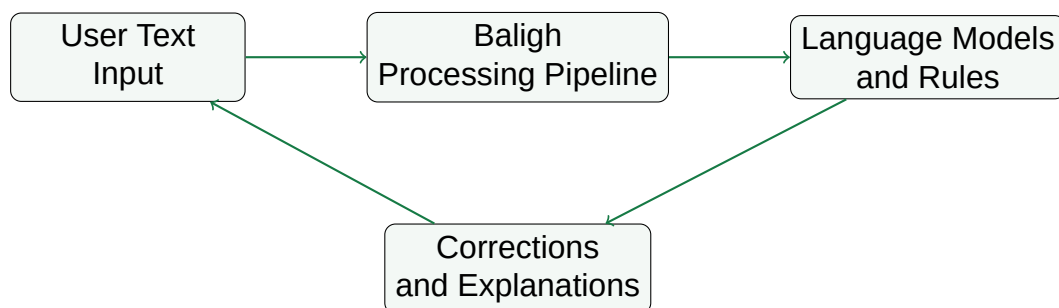


Figure 4.1: High-level architecture of the Baligh system.

4.3. Module 1: Grammatical Error Detection

4.3.1. Functional Description

Explain what this module does.

4.3.2. Modular Decomposition

Break the module into submodules.

4.3.3. Design Constraints

Explain constraints such as latency, Arabic ambiguity, model size, and data availability.

4.3.4. Other Description of Module 1

Add any other relevant technical details.

4.4. Module 2: Correction and Explanation Generation

4.4.1. Functional Description

Explain correction generation and explanation generation.

4.4.2. Modular Decomposition

Break the module into submodules.

4.4.3. Design Constraints

Explain constraints such as explanation quality, grammatical accuracy, and user readability.

4.4.4. Other Description of Module 2

Add any other relevant technical details.

Chapter 5: System Testing and Verification

To ensure that Baligh operates correctly, achieves high linguistic accuracy, and maintains responsive real-time performance, a multi-layered testing and verification strategy was implemented. The validation framework includes automatic unit testing of isolated backend algorithms, end-to-end automated browser testing of frontend interactive elements, and evaluation of core Natural Language Processing (NLP) models on gold-standard Arabic corpora. This chapter details the testing environment setup, the modular and integrated testing methodologies, the continuous integration schedule, and the quantitative evaluation results of the grammatical error detection and next-word prediction modules.

5.1. Testing Setup

Testing and evaluation were conducted in a standardized environment to ensure the reproducibility of all results. The hardware and software specifications of the testing environment are defined as follows:

- **Operating System:** Linux (Ubuntu 22.04 LTS / Debian-based distributions).
- **Python Environment:** Python 3.11.x, managed using the `uv` package installer and virtual environment manager to guarantee deterministic dependency resolution.
- **Node.js Environment:** Node.js v24.x, utilizing the `pnpm` package manager (version 10.33.4) for frontend dependency caching and management.
- **Linguistic Libraries and Resources:**
 - `camel-tools` (version 1.5.7+) loaded with the Modern Standard Arabic morphology database (`morphology-db-msa-r13`) and the Maximum Likelihood Estimation disambiguator (`disambig-mle-calima-msa-r13`).
 - `farasapy` (version 0.1.1) for initial text segmentations.
- **Testing Libraries:** `pytest` (version 8.3.5) for Python backend testing; standard Node testing libraries for frontend unit tests; and `playwright` for automated end-to-end browser execution.

- **Evaluation Corpora:**

- **Qatar Arabic Language Bank (QALB):** The QALB-2014 and QALB-2015 test datasets, which contain native and non-native Arabic comments extracted from Aljazeera articles, characterized by a high frequency of spelling and punctuation errors.
- **ZAEBUC:** A bilingual Arabic-English student writing corpus used to evaluate detection and correction robustness in educational settings.
- **NWP Corpus:** An Arabic text corpus combining Arabic Wikipedia articles and the MAD corpus (totaling 900,000 test-split words) for next-word prediction models.

5.2. Testing Plan and Strategy

The verification strategy relies on testing components in isolation (module testing) before testing their interactions in a fully assembled pipeline (integration testing).

5.2.1. Module Testing

Each subsystem in Baligh is validated independently using automated unit tests to guarantee functional correctness under diverse inputs.

- **Preprocessing Service:** Unit tests in the `tests/services/preprocessing/` directory verify normalizer rules (such as Alif and Ya normalization), sentence boundary detection, word segmenters, and morphological analyzers. For example, `test_analyzer.py` validates that out-of-vocabulary (OOV) tokens (e.g., typos like “الطلاب” containing an extra letter) are correctly flagged with an `is_oov = True` attribute to prevent cascading errors in downstream grammar checks.
- **Grammatical Error Detection (GED):**
 - *Rule-Based Subsystem:* Unit tests verify syntactic, semantic, punctuation, and orthographical rules. Files like `test_syntax.py` and `test_punctuation.py` check specific grammar patterns, such as subject-verb agreement or missing spaces before punctuation.
 - *Lexicon Subsystem:* Verified in `test_detector.py` and `test_trie_runtime.py` to ensure that dictionary words and entity phrases are loaded correctly into MARISA trie data structures for rapid OOV detection.
 - *Machine Learning Subsystem:* Verified in `test_features.py` and `test_detector.py` to ensure sequence labeling features are extracted and parsed correctly

by the CRF model.

- *Fusion Layer*: Tested in `test_fusion.py` to ensure overlap resolution works according to defined tier priorities (e.g., Tier 1 rules override Tier 3 statistical detections; overlapping spans of the same category are merged; duplicate sources are consolidated).
- **Grammatical Error Correction (GEC)**: Evaluated in `tests/services/ged/`. Tests cover ontology-guided candidate generation, alternative ranking engines, spell checking dictionary lookups, and morphological generation.
- **Next-Word Prediction (NWP)**: Verified in `tests/services/nws/` to ensure cache managers, request/response schemas, and character-level N-gram completions run correctly.
- **Frontend Application**: Frontend unit tests ensure utility functions, schema parsers, and editor states function correctly under standard input conditions.

5.2.2. Integration Testing

Integration testing validates that data flows correctly across the entire application stack.

- **GED Orchestrator Integration**: Tested in `test_ged_orchestrator.py`. This ensures that a raw Arabic sentence is properly normalized, segmented, analyzed morphologically, routed through all three detectors, and fused into a final list of resolved error spans.
- **Client-Server API Integration**: Verified by testing backend endpoints to ensure HTTP requests correctly accept Arabic payloads, apply the processing pipeline, and return JSON responses conforming to the API schemas.
- **End-to-End Browser Automation**: Using Playwright (`pnpm test:e2e`), automated Chromium browser instances simulate actual typing sessions. The E2E suite verifies that when a user types text in the editor, error spans are visually highlighted, suggestion dropdowns populate with correction candidates, and explanatory text bubbles appear beneath active cursors.

5.3. Test Schedule

Testing is integrated into the daily development workflow through three distinct stages:

1. **Local Pre-Commit Verification**: Code formatting (`ruff format`), linting checks (`ruff check`), and static type checking (`mypy`) are run on local changes prior to

committing.

2. **Local Pre-Push Validation:** Developers run the complete check suite locally using a unified command (`make all`), which triggers formatting, linting, type-checking, and the full backend `pytest` test suite.
3. **Continuous Integration (CI):** Managed via GitHub Actions on every pull request and push to the `main` branch. The CI workflow splits verification into two parallel jobs:
 - `quality`: Sets up a Python 3.11 environment, installs `astral uv`, restores cached NLP datasets (Farasa and CAMEL Tools), overrides PyTorch with a CPU-only version to reduce runner overhead, downloads morphology databases, and executes formatting checks, linting, type checking, and unit tests.
 - `frontend`: Sets up Node.js 24 and `pnpm`, checks formatting and linting, runs frontend unit tests, installs Playwright dependencies, and executes end-to-end browser tests.

5.4. Comparative Results to Previous Work

The performance of Baligh's primary modules was quantitatively evaluated on established benchmarks.

5.4.1. Grammatical Error Detection (GED) Performance

The GED system was evaluated on QALB-2014, QALB-2015 (separate native L1 and non-native L2 sets), and ZAEBUC test sets. Performance is measured using Precision (P), Recall (R), F1-Score, and False Positives per 1000 tokens (FP/1000). The results are summarized in Table 5.1.

Table 5.1: Grammatical Error Detection Subsystem Performance

Dataset	System / Detector	Precision (P)	Recall (R)	F1-Score	FP/1000
QALB-2014	Rule-Based Detector	69.22%	18.76%	29.52%	18.88
	Lexicon Matcher	89.64%	13.83%	23.96%	3.62
	Sequence Labeler (ML)	92.91%	77.49%	84.50%	13.39
	Fused Orchestrator	92.03%	79.75%	85.45%	15.63
QALB-2015 L1	Rule-Based Detector	71.61%	20.34%	31.68%	17.38
	Lexicon Matcher	88.21%	14.97%	25.59%	4.31
	Sequence Labeler (ML)	92.47%	81.84%	86.83%	14.37
	Fused Orchestrator	91.52%	84.30%	87.76%	16.84
QALB-2015 L2	Rule-Based Detector	55.79%	7.59%	13.36%	13.57
	Lexicon Matcher	88.87%	9.48%	17.13%	2.68
	Sequence Labeler (ML)	82.67%	40.66%	54.51%	19.24
	Fused Orchestrator	82.49%	42.74%	56.31%	20.47
ZAEBUC	Rule-Based Detector	62.58%	19.93%	30.23%	33.41
	Lexicon Matcher	96.97%	8.92%	16.34%	0.78
	Sequence Labeler (ML)	87.63%	77.98%	82.52%	30.87
	Fused Orchestrator	87.58%	79.09%	83.12%	31.46
Aggregate	Rule-Based Detector	68.48%	17.39%	27.73%	17.95
	Lexicon Matcher	89.17%	13.22%	23.02%	3.60
	Sequence Labeler (ML)	91.29%	72.48%	80.81%	15.51
	Fused Orchestrator	90.50%	74.73%	81.86%	17.59

The empirical results show that while the rule-based and lexicon detectors exhibit high precision (especially the lexicon matcher on ZAEBUC at 96.97

5.4.2. Next-Word Prediction (NWP) Performance

The Next-Word Prediction (NWP) models were evaluated on the test split of the LSTM Corpus. We compare the standalone LSTM model against a hybrid predictor that blends LSTM and Word N-Gram models (with confidence weight $\alpha = 0.75$). The evaluation metrics include Top-1, Top-3, and Top-5 prediction accuracy, as shown in Table 5.2.

Table 5.2: Next-Word Prediction (NWP) Accuracy Comparison

Model	Top-1 Accuracy	Top-3 Accuracy	Top-5 Accuracy
Standalone LSTM	10.88%	17.56%	21.52%
Hybrid Predictor	11.40%	18.20%	23.00%

The hybrid predictor outperforms the standalone LSTM model by leveraging high-frequency local word patterns from the N-Gram model alongside the long-range semantic representation of the LSTM.

5.4.3. Word Auto-Completion (WAC) Performance

The context-aware Character N-Gram model for Word Auto-Completion was evaluated under typing scenarios. It achieves a Top-1 accuracy of 30.00%, Top-3 accuracy of 40.80%, and Top-5 accuracy of 46.40%, with a Mean Reciprocal Rank (MRR) of 0.3699 and a Keystroke Savings Rate (KSR) of 7.40%. The performance improves significantly as the length of the typed prefix increases, as detailed in Table 5.3.

Table 5.3: WAC Top-5 Accuracy by Typed Prefix Length

Prefix Length (chars)	1	2	3	4	5	6+
Top-5 Accuracy	11.76%	32.79%	54.00%	77.50%	94.44%	100.00%

As the user types, the character prefixes restrict the possible lexical search space, causing the top-5 prediction accuracy to rise from 11.76% for a single character to 100.00% once six characters have been entered.

Chapter 6: Conclusions and Future Work

6.1. Faced Challenges

Discuss technical, management, research, data, and integration challenges.

6.2. Gained Experience

Discuss technical and non-technical learning outcomes.

6.3. Conclusions

Summarize what the project achieved and why it matters.

6.4. Future Work

Mention possible future improvements and extensions.

References

[1] Author Name, "Paper or Book Title," *Conference or Journal Name*, Year.

[2] Author Name, "Another Reference Title," Publisher or Venue, Year.

Chapter A: Development Platforms and Tools

List programming languages, frameworks, libraries, operating systems, hardware, datasets, and development tools.

Chapter B: Use Cases

Describe system use cases, actors, preconditions, main flows, alternative flows, and expected results.

Chapter C: User Guide

This appendix should be included. Explain how to install, run, and use the final system. Add screenshots or step-by-step instructions when available.

Chapter D: Code Documentation

Document important modules, classes, APIs, configuration files, and code structure.

Chapter E: Feasibility Study

This appendix should be included. Add market, technical, operational, and financial feasibility details.